CASE STUDY

Node.js, CVE-2026-111497: Version Check and Fix in the Lab

A TeamStarWolf case study. Published 4 November 2026; every source checked as of that date.

Executive summary

If you run a Node.js service that accepts compressed request bodies, or a Node.js client that fetches compressed responses, this one is yours. CVE-2026-111497 is an out-of-bounds write in the Google Brotli decoder [1], the compression library that Node.js bundles for `node:zlib` [3], and that .NET, Envoy, Istio, Bun, the Python bindings and a long list of other products also carry [2]. A crafted Brotli stream can corrupt the decoder's memory, and the CVE record says that can lead to memory disclosure, denial of service or possibly code execution [1]. The upstream fix, a small change to how one decoder function reports an error, sat on Brotli's default branch (`master`) from February 2026 until security releases shipped on 21 October 2026 [7] [6]. CISA added it to the Known Exploited Vulnerabilities catalog six days later [9].

The researchers who wrote it up call it Node2Shell (**Reported**: Imperva, ELLIO) [16] [17]. This study is lab-backed. On 4 November 2026 a lab host ran two Express 5 containers that differed only in the Node.js runtime, 24.22.0 against 24.22.1, and sent each the same read-only probe from a public scanner [27]. The verdicts differed: the scanner flagged the 24.22.0 target and not the 24.22.1 one, with different responses to the same probe (**Demonstrated**) [27]. The lab also turned up the most useful practical finding in this study: the Brotli version that Node.js reports about itself is `1.2.0` on the vulnerable runtime and on the fixed one. A version check that trusts the Brotli string will flag a fixed Node.js host as vulnerable and will never see the upgrade land (**Inferred** from the lab result and the CVE's 1.0.3 to 1.2.0 range). Only `node --version` tells them apart (**Demonstrated**) [27].

> Evidence grade: the vulnerability, the fixed versions and the KEV listing are **Confirmed**; the remote code execution write-ups and the in-the-wild traffic report are **Reported**; the lab's version check, differing probe verdicts and the fixed-runtime result are **Demonstrated**.

Key facts at a glance

Field	Value
Identifier	CVE-2026-111497; GitHub advisory GHSA-9v8h-qqgm-4m35; researcher name Node2Shell (**Reported**) [1] [8] [16]
Root component	Google Brotli decoder, `c/dec/decode.c`, versions 1.0.3 through 1.2.0 [1]
Weakness	CWE-787 Out-of-bounds Write and CWE-252 Unchecked Return Value (CNA and NVD) [1] [2]
Scores	NVD CVSS 3.1 9.8 Critical; Google (CNA) CVSS 4.0 9.2 Critical; Node.js rates it High [2] [1] [3]
Access required	Network, no privileges, no user interaction (NVD vector) [2]
Fixed Brotli	1.0.10, 1.1.1 and 1.2.1, released 21 October 2026 [1] [6]
Fixed Node.js	v22.23.4 (LTS), v24.22.1 (LTS) and v26.12.1 (Current), released 21 October 2026 [3] [5]
Other products	.NET, Envoy, Istio, Bun, PyPI brotli and brotlicffi, Go andybalholm/brotli, Debian 12, and others listed by NVD [2] [8]
Exploitation	In CISA KEV since 27 October 2026, due 10 November 2026; CISA's SSVC entry records exploitation as active [9] [1]
Public exploit reports	Remote code execution on the official Node 24.22.0 arm64 image (Imperva) and on unmodified official x86-64 builds (ELLIO) (**Reported**) [16] [17]
Lab result	node 24.22.0 flagged VULNERABLE, node 24.22.1 NOT VULNERABLE, same app; both report Brotli `1.2.0` (**Demonstrated**) [27]
Workaround	Node.js lists none; upgrade the runtime [3]

Key findings

- **The bug is a decoder that did not stop.** An invalid block switch was treated as a temporary shortage of input rather than a permanent format error, so the streaming decoder went on filling an eight-byte internal buffer past its end (**Confirmed**: Node.js) [3]. The upstream fix makes the block-switch decoder return a named format error, `BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH`, instead of a bare `false` its callers ignored, which this study reads as the unchecked return value of CWE-252 (**Inferred**) [7] [1].
- **The fix waited eight months for a release.** The fix commit landed on Brotli's default branch (`master`) on 27 February 2026 with no release. It shipped in Brotli 1.2.1 (as cherry-pick `b418fa81`) and in the Node.js security releases on 21 October 2026, and the CVE was published the next day [7] [6] [8] [12] [5] [1].
- **The Brotli version string cannot confirm the Node.js fix.** Node.js 24.22.1 did not bump its bundled Brotli. It cherry-picked the one upstream commit, so `process.versions.brotli` still reads `1.2.0` (**Confirmed**: Node.js release notes [4]; **Demonstrated**: lab [27]).
- **The lab saw a different answer, not a crash.** The same 23,068,764-byte probe drew HTTP 200 from the vulnerable runtime and HTTP 400 with a block-switch decoder error from the fixed one. The vulnerable container stayed up. The lab plan had expected it might crash; it did not (**Demonstrated**) [27].
- **The fix is the runtime, not the middleware.** The lab's Express 5.3.0 resolved body-parser 2.4.0, newer than the 2.3.0 that Imperva tested as the latest release, and the vulnerable runtime was still flagged, which fits Imperva's own statement that the vulnerable code is in Node.js (**Inferred** from the lab, Imperva and the

Node.js advisory) [27] [16] [3].

- **CISA says it is being exploited.** CISA lists it as known exploited [9], and ELLIO reports a 23 MB Brotli body aimed at Node 24.22.0 on 26 October 2026 whose final stage failed in ELLIO's own testing (**Reported**) [17].

Top recommendations

1. Upgrade Node.js to v22.23.4, v24.22.1, v26.12.1 or later, and rebuild every image built `FROM` an older `node` tag. On 4 November 2026 the download page listed v24.22.2 and v26.13.0 as the current LTS releases [3] [18].
2. Check Node.js with `node --version`, never with `process.versions.brotli`. The Brotli string reads `1.2.0` on vulnerable and fixed runtimes alike [27] [4].
3. Inventory the other copies of Brotli: the system `libbrotli`, PyPI `brotli` and `brotlicffi`, Go `andybalholm/brotli`, .NET runtimes and the proxies in front of your services. Each has its own fixed version (Part 5) [2] [8] [11] [14].
4. Treat it as a KEV item with a date. CISA's due date is 10 November 2026 [9]; plan to meet it whether or not your organization is bound by it.
5. Do not count on crash or error logs to catch a successful attack. In the lab, the block-switch decoder error appeared only on the patched host, and the vulnerable host logged no error for the probe (**Demonstrated**, 02-test.txt) [27].

Scope and method

This study covers CVE-2026-111497 from the upstream fix commit (February 2026) through the lab run on 4 November 2026. It concentrates on Node.js, where the lab ran, and summarizes the other affected products from the NVD record and the vendor advisories that were opened. Every source in the Sources table was opened on 4 November 2026. Vendor advisories for Envoy, Istio, Bun, Siemens and the other NVD-listed products were not opened; their version ranges come from the NVD record only.

Grade	Meaning
Confirmed	A primary source was opened: the CVE record, NVD, the Node.js or Brotli project, a vendor or distribution advisory, or CISA
Reported	A secondary source's account, not independently corroborated (research write-ups, the traffic report, a mailing-list reply)
Demonstrated	Shown in this study's lab run; the capture file is named
Inferred	The study's reasoning from the record and the lab, labelled where it appears

Exploit-class lab steps, the scanner's probe and its invocation, are described in one sentence and never quoted or shown. Version checks and the observables that differ between the two targets are quoted from the captures.

Part 1: the flaw

Where it lives

Brotli is a general-purpose lossless compression format, specified in RFC 7932 (**Confirmed**) [28]. HTTP carries it as the registered `br` content coding, which a sender names in the `Content-Encoding` header (**Confirmed**) [29] [30]. Google's reference implementation is a C library, and its decoder lives in `c/dec/decode.c` [1]. Node.js ships that library inside the runtime and exposes it through `node:zlib`, so any Node.js code that decompresses Brotli data, its own or a framework's, runs Google's decoder [3].

A Brotli stream is a header followed by a series of meta-blocks, and inside a meta-block the data can switch between block types. Each block-switch command is a pair of a block type and a block count, and the decoder reads a new pair from the stream when the current count runs out (**Confirmed**: RFC 7932) [28]. The CVE record describes the result: an out-of-bounds write that lets a remote unauthenticated attacker corrupt adjacent decoder state, disclose memory, cause denial of service or potentially execute code with a crafted stream (**Confirmed**: CNA) [1]. The CNA's title is "Out-of-Bounds Write in Google Brotli Decoder" [1].

The mechanism

Node.js describes it in one sentence (**Confirmed**) [3]. When the decoder met an invalid block switch, it reported a temporary need for more input instead of a permanent format error. A streaming decoder that thinks it needs more input asks for more and keeps appending it, and here it kept appending to an eight-byte internal buffer. That is the out-of-bounds write.

The fix commit shows why it happened (**Confirmed**: Brotli commit `0ed4268`) [7]. The function that decodes the block type and length, `DecodeBlockTypeAndLength`, returned a plain boolean, and its callers did not act on a `false`. The fix changes it to return a decoder error code, `BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH`, so an invalid switch now ends the stream with an error. The commit touches `c/dec/decode.c` and the public header `c/include/brotli/decode.h`, and its subject is "Detect and report invalid stream earlier." [7] [6]. The two weaknesses the CNA lists match the two halves of this: the unchecked return value (CWE-252) is the cause and the out-of-bounds write (CWE-787) is the effect [1] [2] (**Inferred**).

ELLIO dates the bug to Brotli 1.0.3, released in March 2018 (**Reported**) [17]. The Debian security tracker names commit `35e69fc` in v1.0.3 as the change that introduced it [12]. The CVE record's affected range starts at the same release [1].

How an attacker reaches it

The Node.js advisory's condition is simple: the flaw is reachable when an application decompresses attacker-supplied Brotli data with `node:zlib` (**Confirmed**) [3]. The research write-ups name the common ways that happens (**Reported**: Imperva) [16]:

- **Server side.** Express 5.0.0 to 5.3.0 and body-parser 2.0.0 to 2.3.0 decode `br` request bodies in Brotli-capable parsers such as `json()` and `raw()`. A request to such a route with `Content-Encoding: br` hands the body straight to the decoder.
- **Client side.** Node's built-in `fetch()` decodes `br` responses, so a Node.js client that fetches from an attacker-controlled server is exposed.
- **Not by itself.** A bare `node:http` server does not decode incoming Brotli bodies on its own; the application or a framework has to.

Imperva tested body-parser up to 2.3.0, the latest release when it tested on 22 October, and itself places the flaw in Node.js, with the middleware versions only identifying paths that can invoke it (**Reported**) [16]. The lab's body-parser 2.4.0 on the vulnerable runtime was still flagged (Part 4), which extends the write-up rather than contradicting it (**Demonstrated**) [27]. On .NET, the vendor names the same two directions: ASP.NET Core request decompression with `br`, and `HttpClient` automatic decompression that includes Brotli, plus direct use of `BrotliStream` or `BrotliDecoder` (**Confirmed**: .NET advisory) [14].

The fix in each project

Project	Fixed in	How	Source
Google Brotli	1.0.10, 1.1.1, 1.2.1 (21 October 2026)	Security releases whose notes list only this CVE; 1.2.1 carries the fix as cherry-pick `b418fa81` of `0ed4268`	[6] [7] [8] [12]
Node.js	v22.23.4 "Jod" (LTS), v24.22.1 "Krypton" (LTS), v26.12.1 (Current), 21 October 2026 around 10:38 UTC	Two commits in each release: `deps: brotli: cherry-pick 0ed4268a4401abcb44720963` and `src: expose BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH` (v24.22.1: `49dff6f57a`, `b5f41529df`)	[5] [4]

Node.js credits OpenAI Codex Security (Khai Tran and Sylvie Mayer) with the report, and mcollina and aduh95 with the fix (**Confirmed**) [3]. The CVE record credits the same finders [1]. Imperva's write-up says the advisory credits Matteo Collina with reporting; the advisory text does not say that, and this study follows the advisory [3] [16]. The same Node.js release fixed nine other CVEs (CVE-2026-105481, -105482, -105483, -106069, -106070, -106071, -106072, -106073 and -106074) and updated dependencies, so the upgrade carries more than this fix [3].

Part 2: the public record

Timeline

Date	Event	Grade	Source
March 2018	Brotli 1.0.3 introduces the bug	Reported	[17]
27 February 2026	Fix commit `0ed4268` ("Detect and report invalid stream earlier.") lands on Brotli's default branch (`master`); no release	Confirmed	[7]
14 October 2026	Node.js pre-announces a security release for 21 October	Reported	[15]
16 October 2026	Google reserves CVE-2026-111497	Confirmed	[1]
21 October 2026	Node.js v22.23.4, v24.22.1 and v26.12.1 and the security releases post; Brotli 1.0.10, 1.1.1 and 1.2.1	Confirmed	[3] [5] [6]
22 October 2026	CVE published (13:35 UTC); NVD record published; GHSA-9v8h-qqgm-4m35 published; Imperva write-up	Confirmed; Imperva Reported	[1] [2] [8] [16]
23 October 2026	Debian DLA-4855-1 for Debian 12; Ubuntu publishes its CVE page; CISA's SSVC entry records exploitation as active; an oss-security reply points out that the root cause is in Google's library, not only Node.js	Confirmed; oss-security reply Reported	[11] [13] [1] [15]
25 October 2026	.NET advisory with fixed runtimes; ELLIO's first report; the public scanner the lab used is committed	Confirmed; ELLIO Reported; scanner Demonstrated	[14] [17] [27]
26 October 2026	ELLIO updates its report with a 23,069,983-byte Brotli body seen at 15:08 UTC	Reported	[17]
27 October 2026	CISA adds it to KEV as "Google Brotli Unchecked Return Value Vulnerability", due 10 November	Confirmed	[9] [10]
3 November 2026	NVD modifies the record (its latest change when the sources were opened on 4 November); KEV feed catalog version 2026.11.03	Confirmed	[2] [9]
4 November 2026	Lab run: version check, differing probe verdicts, fixed runtime verified, cleanup verified	Demonstrated	[27]
10 November 2026	KEV due date	Confirmed	[9]

The gap that matters is the one between the fix commit and the release. The fix was public on the default branch (`master`) of a widely used library for 236 days before a release carried it to users [7] [6]. Anyone diffing the branch could have seen it. Nothing in the opened record says anyone did before October (**Inferred**: the record is silent, not negative).

Scores and ratings

The scores disagree, and all of them are on the record:

Rater	Rating	Source
NVD (Primary)	CVSS 3.1 9.8 Critical, `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	[2]
Google (CNA)	CVSS 4.0 9.2 Critical, `CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`	[1]
GitHub advisory	Critical (CVSS 3.1 9.8, CVSS 4.0 9.2)	[8]
Node.js	High	[3]
.NET	High, CWE-787	[14]
Ubuntu	Priority High, marked as in CISA KEV	[13]
CISA SSVC (23 October)	Exploitation active, Automatable no, Technical Impact total	[1]

The CNA's vector carries `AT:P`, attack requirements present, which fits the write-ups: the attacker needs a route or client that actually decodes Brotli (**Inferred**) [1] [16]. An oss-security reply on 23 October questioned Ubuntu's earlier "medium" rating [15]; Ubuntu's page now shows High [13].

Who else carries the decoder

NVD's configuration lists products with fixed boundaries, and the vendors this study opened agree with it where they overlap (**Confirmed**: NVD [2]; vendor pages as named):

Product	Vulnerable range (NVD unless noted)	Fixed	Source
Google Brotli	1.0.3 through 1.2.0	1.0.10, 1.1.1, 1.2.1	[1] [6]
Node.js	below 22.23.4; 23.0.0 to below 24.22.1; 25.0.0 to below 26.12.1	22.23.4, 24.22.1, 26.12.1	[2] [3]
PyPI `brotli`	1.0.3 to below 1.2.1 (GHSA)	1.2.1	[8]
PyPI `brotlicffi`	1.0.3 to below 1.2.1.0 (GHSA)	1.2.1.0	[8]
Go `andybalholm/brotli`	below 1.2.7	1.2.7	[2]
.NET	8 below 8.0.33; 9 below 9.0.22; 10 below 10.0.14	8.0.33, 9.0.22, 10.0.14, 11.0.0-rc.3.26522.148 (.NET advisory)	[2] [14]
Envoy	below 1.37.8; 1.38.0 to below 1.38.6; 1.39.0 to below 1.39.3	1.37.8, 1.38.6, 1.39.3	[2]
Istio	below 1.29.9; 1.30.0 to below 1.30.6; 1.31.0 to below 1.31.2	1.29.9, 1.30.6, 1.31.2	[2]
Bun	below 1.4.4	1.4.4	[2]
Debian 12	listed	brotli 1.0.9-2+deb12u1 (DLA-4855-1)	[2] [11]

Product	Vulnerable range (NVD unless noted)	Fixed	Source
Also listed by NVD	edk2, woff2, DuckDB, ClickHouse, Apache Arrow, 7-Zip-zstd, p7zip, and Siemens RUGGEDCOM and Industrial Edge products	see the vendor	[2]

The oss-security thread also cites Envoy's advisory, GHSA-3wvj-j6qm-62mr [15]; it was not opened for this study. Node.js says end-of-life release lines are always affected when it ships a security release, without naming them [3]. NVD's ranges cover all of 23.x and 25.x and every line older than 22, so hosts on those lines reach a fix only by moving to a supported line (**Inferred** from [2] and [3]).

Part 3: exploitation

Three kinds of evidence exist, and they carry different weights.

The listings (Confirmed). CISA added the CVE to the Known Exploited Vulnerabilities catalog on 27 October 2026 with a due date of 10 November 2026, CWE-252 and CWE-787, and known ransomware campaign use marked Unknown [9] [10]. The catalog's required action is to apply vendor mitigations per BOD 26-04 [10]. Four days earlier, CISA's Vulnrichment SSVC entry in the CVE record had already marked exploitation as active and technical impact as total, with automatable marked no [1]. KEV and SSVC are CISA's judgments; neither names an actor or a victim.

The research reproductions (Reported). Imperva's Ron Masas reported remote code execution against the official `node:24.22.0` Linux arm64 image on 22 October; it took multiple attempts, and 24.22.1 rejected the same inputs [16]. ELLIO reports reproducing remote code execution on unmodified official x86-64 Node builds with ASLR on, using an offset specific to each build. It adds that failed attempts crash worker processes, a denial of service in themselves, and that the resulting core dumps may hold secrets [17]. Imperva makes no claim of exploitation in the wild [16]. Neither exploit was examined for this study.

The traffic (Reported). ELLIO Threat Research Lab reports a scanner that calls itself `node2shell-check`, from a source in AS44620 that its sensors first saw on 13 June 2026, followed on 26 October 2026 at 15:08 UTC by a 23,069,983-byte Brotli body aimed at Node v24.22.0 on x86-64 [17]. ELLIO says the payload's final stage failed in its own testing [17]. The June date is when that source address first appeared, not evidence that this CVE was being probed in June (**Inferred**).

The honest summary: CISA says it is exploited, two research teams say code execution works on stock builds with effort, and one sensor network saw an attempt whose payload ELLIO could not get to work in its own testing. That is enough to patch on the KEV clock. It is not evidence of a mass-scanning wave of the PHP2Shell kind, which the PHP2Shell study covers for the same month.

Part 4: what the lab showed

The lab ran on a lab host on 4 November 2026 UTC, with every target bound to loopback and every container removed afterwards [27]. The screenshots are renderings of the captured terminal output with a neutral `lab$` prompt and the lab host anonymized. The test step itself, the scanner's probe, is described below and not shown.

Setup: two targets that differ only in the runtime

Both targets were built from the same Dockerfile and the same two-route Express 5 app, `server.js`, whose body parser is `express.json({ limit: '100mb' })` [27] (01-setup.txt). One image was built `FROM node:24.22.0` (digest `sha256:925de64a...f017f5`) and the other `FROM node:24.22.1` (digest `sha256:d862bb5d...108ea`) (**Demonstrated**, 01-setup.txt). `npm ls` in each container resolved the same tree, `express@5.3.0` with `body-parser@2.4.0` (**Demonstrated**, 01-setup.txt and 03-exposure.txt). Each container published its port on `127.0.0.1` only, and each printed its banner:

```
LAB-READY node=v24.22.0 brotli=1.2.0
LAB-READY node=v24.22.1 brotli=1.2.0
```

(**Demonstrated**, 01-setup.txt)



The setup did not go as planned, and the capture records it (**Demonstrated**, 01-setup.txt). The first image builds failed: npm stopped with `EAI_AGAIN` and pip with name-resolution errors, because the build containers could not reach the lab's private resolver. The retry resolved the three public registry hosts on the lab host and passed them to the builds with `--add-host`. The scanner's Dockerfile base was changed from `python:3.13-slim` to `python:3.12-slim` in a separate `Dockerfile.lab`, and the working directory was

`/tmp/lab`, not the plan's `~/lab`.

The version check

```
lab$ docker exec n2s-vuln node --version; docker exec n2s-fixed node --version
v24.22.0
v24.22.1
```

Over HTTP, each app's `/version` route reported its runtime and its bundled Brotli (**Demonstrated**, 02-test.txt):

```
{"node":"v24.22.0","brotli":"1.2.0"}
{"node":"v24.22.1","brotli":"1.2.0"}
```

This is the study's key finding. The bundled Brotli version is `1.2.0` on the vulnerable runtime and on the fixed one, so the Brotli string cannot confirm the fix; only the Node.js version can (**Demonstrated**, 02-test.txt). ELLIO reports the same for patched and vulnerable builds (**Reported**) [17]. The reason is in the release notes: Node.js 24.22.1 did not move its bundled Brotli to 1.2.1, it cherry-picked commit `0ed4268` into the copy it already had (**Confirmed**) [4]. A plain JSON control body, `{"a":1}`, was accepted by both targets with `keys:1` (**Demonstrated**, 02-test.txt).

The test step, described

Over loopback, the MIT-licensed Assetnote node2shell-scanner (commit `bf2dc387`, 25 October 2026) sent four one-byte control requests, one 23,068,764-byte Brotli probe with `Expect: 100-continue` and a final one-byte control to `/api/json` on each target, the fixed target first (**Demonstrated**, 02-test.txt, fixed.json, vuln.json). The lab plan classes the scanner as a read-only probe, and no code execution was attempted (plan.json).

What differed

Observable	node 24.22.1 (fixed)	node 24.22.0 (vulnerable)	Capture
Scanner verdict	`NOT VULNERABLE`, `NO SIGNAL`	`VULNERABLE`	02-test.txt, fixed.json, vuln.json
Probe response	HTTP 400, `text/html`, 359 bytes (the malformed-Brotli class)	HTTP 200, `application/json`, 38 bytes (the valid-route class)	fixed.json, vuln.json
Final control	HTTP 200	HTTP 200	fixed.json, vuln.json
Last application log entry	`Error: Decompression failed`, code `ERR__ERROR_FORMAT_BLOCK_SWITCH`, errno -17, status 400, from `BrotliDecoder.zlibOnError`	No error for the probe; the last entry is the unsupported-encoding `x` control	02-test.txt
Container state after the probe	`Up`	`Up`	02-test.txt, 04-mitigation.txt

(**Demonstrated** throughout.) The fixed runtime treated the probe as a malformed stream and said so with the error code the Node.js fix exposes, `BROTLI_DECODER_ERROR_FORMAT_BLOCK_SWITCH` [4]. The vulnerable

runtime accepted the same bytes as if they were a valid body. The log entries were attributed by counting the lines of the two `docker logs --tail 20` outputs (02-test.txt).

What contradicted the plan

No crash. The lab plan inferred that the vulnerable container might exit with code 139 (a segmentation fault) or abort after the probe. It did not. `docker ps -a` showed both `n2s-vuln` and `n2s-fixed` up after the probe and still up about a minute later (**Demonstrated**, 02-test.txt and 04-mitigation.txt). A crash is not a reliable sign that this probe reached a host, and Imperva says crashes and decoder errors are not reliable signs of exploitation on their own (**Reported**) [16].

A newer body-parser, the same result. Imperva tested body-parser 2.0.0 to 2.3.0; 2.3.0 was the latest release when it tested on 22 October, and the write-up itself places the flaw in Node.js, with the middleware versions only identifying paths that can invoke it (**Reported**) [16]. The lab's body-parser was 2.4.0 and the vulnerable runtime was still flagged (**Demonstrated**) [27], so the lab extends the write-up rather than contradicting it. Together with the Node.js advisory's statement that the decoder is the one bundled with Node.js [3], the reading is that the runtime carries the fix, not the body-parser version (**Inferred**) [27].

The exposure check on the lab host

The plan's exposure checks ran on the lab host itself and on the two containers (**Demonstrated**, 03-exposure.txt): the host's Node.js was `v24.21.0` (Brotli `1.2.0`), its system library `libbrotli1:amd64 1.2.0-4+b1`, its PyPI `Brotli 1.2.0`, and its only .NET runtimes 6.0.8. A search of `go.mod` files under the working directory found no `andybalholm/brotli`, and `envoy`, `istioctl`, `bun` and the `brotli` command were absent.

CVE-2026-111497 Exposure check

```
# exposure checks run on the lab host; container loop scoped to this plan's n2s- containers so other plans' containe
lab$  node --version; node -p 'process.versions.brotli'
v24.21.0
1.2.0

lab$  docker exec n2s-vuln node -p 'process.version + " brotli " + process.versions.brotli'; docker exec n2s-fixed r
v24.22.0 brotli 1.2.0
v24.22.1 brotli 1.2.0

lab$  for c in $(docker ps -q --filter name=n2s-); do echo "$c $(docker exec $c node --version 2>/dev/null)"; done
23541318139c v24.22.1
bfc198e71cc4 v24.22.0

lab$  docker exec n2s-vuln npm ls express body-parser 2>/dev/null; docker exec n2s-fixed npm ls express body-parser
app@1.0.0 /app
`-- express@5.3.0
  `-- body-parser@2.4.0

app@1.0.0 /app
`-- express@5.3.0
  `-- body-parser@2.4.0


lab$  dpkg -l | grep -i libbrotli; brotli --version 2>/dev/null
ii  libbrotli1:amd64                          1.2.0-4+b1                              amd64        library impleme

lab$  python3 -m pip show brotli brotlicffi 2>/dev/null | grep -E '^(Name|Version)'
Name: Brotli
Version: 1.2.0

lab$  grep -R 'andybalholm/brotli' --include=go.mod . 2>/dev/null

lab$  dotnet --list-runtimes 2>/dev/null
Microsoft.AspNetCore.App 6.0.8 [/usr/share/dotnet/shared/Microsoft.AspNetCore.App]
Microsoft.NETCore.App 6.0.8 [/usr/share/dotnet/shared/Microsoft.NETCore.App]

lab$  envoy --version 2>/dev/null; istioctl version --remote=false 2>/dev/null; bun --version 2>/dev/null
```

Read against the published ranges, the lab host carried versions inside a vulnerable range in three places (**Inferred**) [2] [8] [12]. Node.js `v24.21.0` falls in NVD's range of 23.0.0 up to but not including 24.22.1. PyPI Brotli `1.2.0` falls in the GitHub advisory's range of 1.0.3 up to but not including 1.2.1. Under dpkg version ordering, `libbrotli1 1.2.0-4+b1` sorts below Debian unstable's fixed `1.2.0-4.1`. The .NET 6.0.8 runtimes fall in no listed range: 6.0 is an out-of-support line that the .NET advisory does not assess, which is not the same as safe. Ubuntu lists its dotnet6 package on 22.04 as vulnerable with the fix deferred [13].

The fix, verified

The only mitigation verified was the fixed runtime (**Demonstrated**, 04-mitigation.txt). The step re-read the fixed container's runtime, `v24.22.1`, and the two saved verdicts: the probe drew 200 with 38 bytes from the vulnerable target and 400 with 359 bytes from the fixed one. No workaround was tested: neither stripping `Content-Encoding: br` at a proxy nor turning off body inflation in the parser.



Cleanup, verified

Both containers and all six images the plan used were removed, `/tmp/lab/CVE-2026-111497` was gone, the check on ports 3001 and 3002 printed `ports clear`, `docker ps -a` was empty, the image check printed `no plan images left`, and the only Docker networks left were `bridge`, `host` and `none` (**Demonstrated**, 05-cleanup.txt).

CVE-2026-111497 Cleanup

```
lab$  docker rm -f n2s-vuln n2s-fixed
n2s-vuln
n2s-fixed

lab$  docker rmi lab-node2shell:vuln lab-node2shell:fixed node2shell-scanner
Untagged: lab-node2shell:vuln
Deleted: sha256:776d6e73ffd3527c96f660580e1c557c29b32ea61b9e7700eb03bd33c6a66419
Untagged: lab-node2shell:fixed
Deleted: sha256:f53d3575c7e43d83b701aa22b351a75d4e152d65af9ba15cc21ef663dbe2149e
Untagged: node2shell-scanner:latest
Deleted: sha256:b89dab044ebef7614b947087416b1f55c5360beab87d5904cb5184cdbab784d3

lab$  docker rmi node:24.22.0 node:24.22.1 python:3.12-slim
Untagged: node:24.22.0
Untagged: node@sha256:925de64a85319052d6971ae81ef2417da956e8a3a2c4a0fac732327970f017f5
Deleted: sha256:fc72e5e902c9c29ad2701a67478f85d0bcbc1e8b20d543d71cfec319a68ae346
Untagged: node:24.22.1
Untagged: node@sha256:d862bb5d082c5ae0210ac07e8c0344cb8cf7bef6d78e9b8c50c179da923108ea
Deleted: sha256:8279a176db0ac2d89d4d3ef16063c1990ee00ea98d81e0937ed38117feab7fed
Untagged: python:3.12-slim
Untagged: python@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30
Deleted: sha256:ae4e8b7c575fabea88a029b6cae757f342ca2525e07f7c22272d5105c346b35e

lab$  rm -rf /tmp/lab/CVE-2026-111497

lab$  ls -d /tmp/lab/CVE-2026-111497 2>&1
ls: cannot access '/tmp/lab/CVE-2026-111497': No such file or directory

lab$  ss -ltnp | grep -E ':(3001|3002)\b' || echo 'ports clear'
ports clear

lab$  docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES     IMAGE     STATUS

lab$  docker ps -a --filter name=n2s- --format '{{.Names}}' | wc -l; docker images --format '{{.Repository}}:{{.Tag}
0
no plan images left

lab$  docker network ls --format '{{.Name}}'
bridge
host
none
```

What the lab does and does not show

It shows, on one host on one date, that an otherwise identical Express 5 app answers the public probe differently on Node.js 24.22.0 and 24.22.1, that changing only the runtime version changes the answer, and that the Brotli version string does not change with it (**Demonstrated**) [27]. It does not show code execution, which was not attempted; it does not cover arm64, the client-side `fetch()` path, .NET or the proxies; and it does not test any workaround. The remote code execution reports stay **Reported** [16] [17].

Part 5: find it and fix it on real hosts

Every check below only reads state. The fixed versions are **Confirmed** from the sources named; the lab ran the Node.js, container, dpkg, pip, Go, .NET and proxy checks on its own host (03-exposure.txt).

Where Brotli lives	How to check	Fixed in	Source
Node.js, any install method	`node --version`; never `process.versions.brotli`	v22.23.4, v24.22.1, v26.12.1 or later; 23.x, 25.x and lines older than 22 have no fix on their own line (**Inferred**)	[3] [2] [27]
Node.js in containers	`docker exec <container> node --version` for each running container; for images, check the `FROM node:` tag	Rebuild on a fixed tag and redeploy; a running container keeps its runtime until replaced	[27] [3]
Express 5 and body-parser	`npm ls express body-parser` shows whether a Brotli-capable parser is in the tree	Not a fix on its own: body-parser 2.4.0 was still flagged on a vulnerable runtime (**Inferred**)	[27] [16]
Debian and Ubuntu system library	`dpkg -l | grep -i libbrotli`	Debian 12 1.0.9-2+deb12u1 (DLA-4855-1); Ubuntu 24.04 1.1.0-2ubuntu0.1 and 22.04 1.0.9-2ubuntu0.1	[11] [13]
Debian 13 and unstable	as above	trixie 1.1.0-2+deb13u1 (DSA-6582-1); forky and sid 1.2.0-4.1 (**Inferred** reading of the Debian tracker)	[12]
Distribution `nodejs` packages	`dpkg -l nodejs`	Debian's `nodejs` uses the system Brotli library, so updating `libbrotli` is what fixes it (**Inferred** from the Debian tracker). Ubuntu lists its `nodejs` package as not affected, except 20.04, listed vulnerable with no fix	[12] [13]
Python	`python3 -m pip show brotli brotlicffi`	`brotli` 1.2.1; `brotlicffi` 1.2.1.0	[8]
Go	search `go.mod` files for `andybalholm/brotli`	1.2.7	[2]
.NET	`dotnet --list-runtimes`	8.0.33, 9.0.22, 10.0.14, 11.0.0-rc.3.26522.148; rebuild and redeploy self-contained apps and republish NativeAOT apps, as the .NET advisory directs; updating the shared runtime does not replace a decoder linked into a NativeAOT executable	[14]
Envoy, Istio, Bun	each product's version command	Envoy 1.37.8, 1.38.6, 1.39.3; Istio 1.29.9, 1.30.6, 1.31.2; Bun 1.4.4 (NVD; vendor advisories not opened)	[2]

Debian's LTS advisory recommends upgrading the brotli packages [11]. On Debian-family hosts that is one package update for every program that links the system library; on hosts running the official Node.js binaries or images it changes nothing for Node.js, which bundles its own copy [3] [12] (**Inferred**). Expect both kinds on the same host: the lab host had a system `libbrotli` and a separately installed Node.js (03-exposure.txt).

If you cannot upgrade today

Node.js lists no workaround (**Confirmed**) [3]. .NET does: remove the `br` provider from ASP.NET Core request decompression and limit `HttpClient` automatic decompression to GZip and Deflate (**Confirmed**: .NET advisory) [14]. For Node.js, the bridges a team might reach for are rejecting or stripping `Content-Encoding: br` on request bodies at the proxy, or turning off body inflation in the parser for routes that do not need it. Neither is vendor-documented for this CVE and neither was tested in the lab (**Inferred**) [27]. Neither covers the client-side `fetch()` path [16]. Treat any of them as a way to buy days before the upgrade, not as the fix.

Part 6: detection

- **Version inventory is the control that works.** Flag Node.js below the fixed lines by `node --version`, system `libbrotli` below the distribution's fixed package, and the language packages below their fixed versions [2] [8] [11]. The lab showed that the Brotli string will not tell you (**Demonstrated**) [27].
- **The public scanner has a recognizable shape.** In the lab it sent a POST to a JSON route with `Content-Encoding: br`, a `Content-Length` of 23,068,764 and `Expect: 100-continue`, preceded from the same source by one-byte `br` controls and a control with the unsupported encoding `x` (**Demonstrated**, fixed.json and vuln.json). That is the scanner's shape; an exploit need not look like it (**Inferred**). ELLIO's in-the-wild body was a similar size, 23,069,983 bytes (**Reported**) [17].
- **Decoder errors mark probes of patched hosts.** On the patched runtime the probe left a 400 and `ERR__ERROR_FORMAT_BLOCK_SWITCH` in the application log; on the vulnerable runtime it left no error and no crash (**Demonstrated**, 02-test.txt). Alerting on that error finds someone probing hosts you have already fixed, not success against the ones you have not (**Inferred**).
- **Crashes are a weak signal both ways.** ELLIO says failed exploit attempts crash workers [17]; the lab's probe did not crash anything [27]; Imperva says crashes and decoder errors are not reliable signs of exploitation on their own [16]. Where a Node.js worker does crash after a large `br` body, keep the core dump under access control: ELLIO warns it may hold secrets [17].

Part 7: the vulnerability-management view

- **A bundled library hides behind the product's own version.** Node.js ships Brotli inside the runtime, so a software inventory keyed on the library version sees `1.2.0`, flags fixed runtimes as vulnerable and never records the fix. The same pattern, a library copied into a product rather than linked, is why Log4Shell defeated inventories built on package names (**Inferred**). For Node.js, the product version is the record that counts [4] [27].
- **One CVE, many patch owners.** The decoder is in the runtime team's Node.js, the platform team's .NET, the network team's Envoy and Istio, the data team's Python and ClickHouse, and the operating-system team's `libbrotli` [2]. Route the ticket per copy, not per CVE (**Inferred**).
- **The scores and the clock agree for once.** A 9.8 base score, a KEV listing with a due date and an SSVC "active" all point the same way [2] [9] [1]. The CNA's `AT:P` and the reachability conditions narrow the emergency to services and clients that decode Brotli from untrusted sources, but finding those takes longer than upgrading the runtime everywhere (**Inferred**).

- **A fix on a branch is not a fix in the field.** The upstream commit sat unreleased for 236 days [7] [6]. Projects that vendor a dependency inherit that dependency's release cadence; watching the upstream security branch, not only its releases, is the earlier signal (**Inferred**).

The library's CVE Reference had no entry for CVE-2026-111497, Brotli or Node2Shell on 4 November 2026 [26].

MITRE mappings

ATT&CK has no procedure example that names this CVE. Each technique below is this study's binding of the record's attack paths to the technique's definition, with the basis stated; every id was checked on the live ATT&CK page and against the library's `data/attack/technique_profiles.jsonl` [19] [20] [21] [26].

ATT&CK Enterprise techniques

ATT&CK id	Name	Tactic (v20)	Dated event or report	Basis	Source
T1190	Exploit Public-Facing Application	Initial Access	NVD record (2026-10-22); Imperva's Express 5 parsers (2026-10-22); ELLIO's traffic to Node 24.22.0 (2026-10-26); lab Express route (2026-11-04)	NVD vector `AV:N/PR:N/UI:N` plus a server-side route that decodes `br` bodies (Confirmed, Demonstrated)	[2] [16] [17] [27] [19]
T1203	Exploitation for Client Execution	Execution	Imperva's `fetch()` path (2026-10-22, Reported); .NET `HttpClient` automatic decompression (2026-10-25, Confirmed)	Client software decoding an attacker-supplied `br` response	[16] [14] [20]
T1499.004	Endpoint Denial of Service: Application or System Exploitation	Impact	CVE record (2026-10-22); ELLIO's crashing workers (2026-10-25, Reported)	The CNA's "cause a denial of service" impact	[1] [2] [17] [21]

ATT&CK for ICS is not bound: NVD lists Siemens Industrial Edge and RUGGEDCOM products, but the record describes no ICS-specific behavior (**Inferred**) [2]. ATT&CK for Mobile does not apply.

Group, software and campaign objects

None is bound. The record attributes the exploitation CISA lists to no named actor, and the traffic ELLIO reports carries only a network (AS44620) and a scanner name [9] [17]. No ATT&CK group, software or campaign object in the opened record is tied to this CVE.

CWE and CAPEC

Id	Name	Where it appears	Source
CWE-252	Unchecked Return Value	CNA, NVD, KEV and the GitHub advisory; the cause, in this study's reading of the fix commit (**Inferred**)	[1] [2] [9] [8] [22]
CWE-787	Out-of-bounds Write	CNA, NVD, KEV and the .NET advisory; the effect	[1] [2] [9] [14] [23]

The GitHub advisory lists only CWE-252, and the .NET advisory only CWE-787 [8] [14]. The library has no per-id CWE pages; see the CWE Reference.

No CAPEC id is bound. In the library's `data/weaknesses/cwe.jsonl`, `related_capec` is empty for both CWE-252 and CWE-787, and neither live CWE page has a Related Attack Patterns section [26] [22] [23]. CAPEC-100 Overflow Buffers relates to CWE-119, CWE-120, CWE-129, CWE-131, CWE-680 and CWE-805, not to CWE-787, so it is left out [24].

D3FEND countermeasures

Each countermeasure below has a page in the library; six of them are mapped in the library's data to T1190 or T1203, and four are this study's selection (**Inferred**) [26] [25]:

Countermeasure	Id	Why it applies here	D3FEND tactic; library mapping
Software Update	D3-SU	The vendor fix: upgrade Node.js, Brotli and the other carriers	Harden; no library mapping to these techniques [25]
Software Inventory	D3-SWI	Finding every copy of the decoder	Model; no library mapping
Asset Vulnerability Enumeration	D3-AVE	Comparing runtime versions with the fixed lines	Model; no library mapping
Inbound Traffic Filtering	D3-ITF	Filtering `br` request bodies at the edge	Isolate; mapped to T1190
Network Traffic Filtering	D3-NTF	As above, between internal tiers	Isolate; mapped to T1190
Segment Address Offset Randomization	D3-SAOR	ASLR; ELLIO needed a build-specific offset to get past it	Harden; mapped to T1190 and T1203
Process Segment Execution Prevention	D3-PSEP	Non-executable memory against a corrupted-heap payload	Harden; mapped to T1190 and T1203
Stack Frame Canary Validation	D3-SFCV	Memory-corruption hardening in the client path	Harden; mapped to T1203
Memory Boundary Tracking	D3-MBT	Catching the out-of-bounds write itself	Detect; mapped to T1203
Application Exception Monitoring	D3-AEM	The `ERR__ERROR_FORMAT_BLOCK_SWITCH` decoder errors, with the limits in Part 6	Detect; no library mapping

Where the last column records a mapping, it is the library's (`data/attack/technique_to_d3fend.jsonl` and `data/attack/d3fend_countermeasures.jsonl`); the choice of the four unmapped countermeasures and the "why it applies" column are this study's reading (**Inferred**), and the ASLR note rests on ELLIO's report (**Reported**) [17]. The library has no D3FEND mapping for T1499.004 [26]. See the D3FEND Reference.

Navigator layer

navigator/library-layers/nodejs-cve-2026-111497.json carries the three technique entries above (ATT&CK v20, domain `enterprise-attack`). Score 100 marks the binding the record and the lab both support (T1190); score 75 marks bindings from the record without a lab demonstration (T1203, T1499.004). Each entry's comment names the source numbers.

MITRE ATT&CK, CWE, CAPEC and D3FEND content is used under the terms recorded in THIRD_PARTY_NOTICES.md.

Cross-cutting themes

- **Check the product, not the part.** The Brotli string reads `1.2.0` before and after the Node.js fix, so the version a scanner or an engineer reads off the component can be the wrong one to trust when a vendor cherry-picks a fix instead of bumping the library [4] [27]. The same lesson runs through Log4Shell and the bundled-dependency cases in the npm supply-chain study (**Inferred**).
- **A quiet fix is a public fix.** An unreleased commit on a public branch is readable by anyone. The 236 days between the Brotli commit and the release were 236 days in which the fix could be diffed [7] [6] (**Inferred**).
- **Absence of a crash is not absence of a hit.** The vulnerable runtime accepted the probe without an error or a crash, while the patched one logged an error. Logging that looks louder on the fixed host is the opposite of what an alert rule usually assumes (**Demonstrated**, with the reading **Inferred**) [27].
- **Lab plans are hypotheses.** The plan expected that the vulnerable container might crash; the run showed it did not. A lab that writes down its expectations first is how that correction reached this study [27].
- **Machine-assisted finding is now in the credits.** The CNA and Node.js credit the report to OpenAI Codex Security, with two named researchers [1] [3]. What the tool did and what the people did is not in the record. The direction matches the AI Threats 2026 study and the CVE-2026-98432 study (**Inferred**).

What is overstated and what is understated

- **Overstated:** "remote code execution on any Node.js server." The flaw is reachable only where an application or framework decodes attacker-supplied Brotli; a bare `node:http` server does not do that on its own, and the CNA's vector carries `AT:P`, which fits that condition (Confirmed [1] [3]; Reported [16]; the link between them Inferred). The two code-execution reports needed multiple attempts or a build-specific offset (Reported [16] [17]).
- **Overstated:** the in-the-wild picture. CISA lists the CVE as known exploited [9], but the one public traffic report describes a single body whose final stage failed in ELLIO's own testing, and no opened record names an actor or a victim (Confirmed [9]; Reported [17]). This is not a mass-exploitation wave of the PHP2Shell kind (Inferred).
- **Understated:** the reach beyond Node.js. The same decoder sits in .NET, Envoy, Istio, Bun, the Python and Go packages, the Debian and Ubuntu system library and the other products NVD lists, each with its own fix and its own owner (Confirmed [2] [8] [11] [14]). The researcher name points at one runtime; the patch work spans many (Inferred).
- **Understated:** the inventory blind spot. The Brotli version string reads `1.2.0` on the vulnerable and the fixed runtime, a body-parser version outside the write-up's range was still flagged, and the patched host

logged a decoder error while the vulnerable one logged nothing (Demonstrated [27]). Checks built on the component version, the middleware version or error logs all point the wrong way here (Inferred).

- **Understated:** how long the fix was public. The upstream commit sat on Brotli's default branch for 236 days before a release carried it (Confirmed [7] [6]; Inferred for the arithmetic).

Sources

Sources 1 to 26 and 28 to 30 were opened on 4 November 2026; source 27 is the lab capture set, recorded on 4 November 2026 UTC.

#	Source	URL	Opened	Grade
1	CVE record CVE-2026-111497 (CNA Google; CISA ADP Vulnrichment SSVC dated 2026-10-23)	https://cveawg.mitre.org/api/cve/CVE-2026-111497	2026-11-04	Confirmed
2	NVD record CVE-2026-111497 (API 2.0; published 2026-10-22, last modified 2026-11-03 as of 4 November)	https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-111497	2026-11-04	Confirmed
3	Node.js, "Wednesday, October 21, 2026 Security Releases"	https://nodejs.org/en/blog/vulnerability/october-2026-security-releases	2026-11-04	Confirmed
4	Node.js v24.22.1 release notes	https://nodejs.org/en/blog/release/v24.22.1	2026-11-04	Confirmed
5	Node.js v24.22.1 release on GitHub (with v22.23.4 and v26.12.1)	https://github.com/nodejs/node/releases/tag/v24.22.1	2026-11-04	Confirmed
6	Google Brotli v1.2.1 release	https://github.com/google/brotli/releases/tag/v1.2.1	2026-11-04	Confirmed
7	Google Brotli commit 0ed4268, "Detect and report invalid stream earlier."	https://github.com/google/brotli/commit/0ed4268a4401abcb4472096340477cf4af7d1a2b	2026-11-04	Confirmed
8	GitHub Advisory GHSA-9v8h-qqgm-4m35	https://github.com/advisories/GHSA-9v8h-qqgm-4m35	2026-11-04	Confirmed
9	CISA Known Exploited Vulnerabilities feed (catalog 2026.11.03)	https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json	2026-11-04	Confirmed
10	CISA KEV catalog entry for CVE-2026-111497	https://www.cisa.gov/known-exploited-vulnerabilities-catalog?field_cve=CVE-2026-111497	2026-11-04	Confirmed
11	Debian LTS advisory DLA-4855-1 (2026-10-23)	https://lists.debian.org/debian-lts-announce/2026/10/msg00049.html	2026-11-04	Confirmed
12	Debian security tracker, CVE-2026-111497	https://security-tracker.debian.org/tracker/CVE-2026-111497	2026-11-04	Confirmed (listing); readings Inferred
13	Ubuntu security, CVE-2026-111497	https://ubuntu.com/security/CVE-2026-111497	2026-11-04	Confirmed

#	Source	URL	Opened	Grade
14	.NET announcement, Brotli decoder advisory (2026-10-25)	https://github.com/dotnet/announcements/issues/494	2026-11-04	Confirmed
15	oss-security thread on CVE-2026-111497 (2026-10-23)	https://www.openwall.com/lists/oss-security/2026/10/23/25	2026-11-04	Reported (mailing-list thread)
16	Imperva, Ron Masas, "Node2Shell: from Brotli data to remote code execution" (2026-10-22)	https://www.imperva.com/blog/node2shell-from-brotli-data-to-remote-code-execution/	2026-11-04	Reported
17	ELLIO Threat Research Lab, Node2Shell CVE-2026-111497 report (2026-10-25, updated 2026-10-26)	https://ellio.tech/en/blog/node2shell-cve-2026-111497-rce-on-stock-nodejs-builds/	2026-11-04	Reported
18	Node.js download page	https://nodejs.org/en/download	2026-11-04	Confirmed
19	MITRE ATT&CK T1190 (version 2.8, last modified 12 May 2026)	https://attack.mitre.org/techniques/T1190/	2026-11-04	Confirmed
20	MITRE ATT&CK T1203	https://attack.mitre.org/techniques/T1203/	2026-11-04	Confirmed
21	MITRE ATT&CK T1499.004	https://attack.mitre.org/techniques/T1499/004/	2026-11-04	Confirmed
22	CWE-252 Unchecked Return Value (CWE 5.0)	https://cwe.mitre.org/data/definitions/252.html	2026-11-04	Confirmed
23	CWE-787 Out-of-bounds Write (CWE 5.0)	https://cwe.mitre.org/data/definitions/787.html	2026-11-04	Confirmed
24	CAPEC-100 Overflow Buffers	https://capec.mitre.org/data/definitions/100.html	2026-11-04	Confirmed
25	MITRE D3FEND, Software Update	https://d3fend.mitre.org/technique/d3f:SoftwareUpdate/	2026-11-04	Confirmed
26	Library data on main: `data/weaknesses/cwe.jsonl`, `data/attack/technique_to_d3fend.jsonl`, `data/attack/d3fend_countermeasures.jsonl`, `data/attack/technique_profiles.jsonl`, the `mitre/` tree and `CVE_REFERENCE.md`	https://github.com/TeamStarWolf/TeamStarWolf	2026-11-04	Confirmed
27	Lab captures for CVE-2026-111497: `plan.json`, `01-setup.txt`, `02-test.txt`, `03-exposure.txt`, `04-mitigation.txt`, `05-cleanup.txt`, `vuln.json`, `fixed.json` and the screenshots in this study	not published	2026-11-04	Demonstrated
28	RFC 7932, "Brotli Compressed Data Format" (July 2016)	https://www.rfc-editor.org/rfc/rfc7932.txt	2026-11-04	Confirmed
29	RFC 9110, "HTTP Semantics", Section 8.4 Content-Encoding	https://www.rfc-editor.org/rfc/rfc9110.txt	2026-11-04	Confirmed
30	IANA HTTP Content Coding Registry (`br` entry, registered 2016-05-24)	https://www.iana.org/assignments/http-parameters/http-parameters.xml	2026-11-04	Confirmed

© 2026 The MITRE Corporation. This work is reproduced and distributed with the permission of The MITRE Corporation. License: ATT&CK Terms of Use. ATT&CK® and MITRE ATT&CK® are registered trademarks of The MITRE Corporation. This project is not affiliated with or endorsed by MITRE.

© The MITRE Corporation. Includes material from the Common Weakness Enumeration (CWE™), used under the CWE Terms of Use. https://cwe.mitre.org/

© The MITRE Corporation. Includes material from the Common Attack Pattern Enumeration and Classification (CAPEC™), used under the CAPEC Terms of Use. https://capec.mitre.org/

© The MITRE Corporation. Includes material from MITRE D3FEND™, used under the D3FEND Terms of Use. https://d3fend.mitre.org/

The terms for every MITRE dataset used here are recorded in THIRD_PARTY_NOTICES.md.

Related references

- CVE Reference
- Notable Incidents
- Threat Actors
- Malware Families
- CWE Reference, D3FEND Reference
- ATT&CK techniques: T1190 Exploit Public-Facing Application, T1203 Exploitation for Client Execution, T1499.004 Application or System Exploitation
- Vulnerability Prioritization Reference, Vulnerability Management Reference
- Express, JavaScript, Debian, Ubuntu
- Case studies: Log4Shell, PHP2Shell and the October 2026 Attack Wave, npm Supply-Chain Attacks, CVE-2026-98432, AI Threats 2026
- D3FEND: all countermeasures
- Navigator layers, including this study's layer, nodejs-cve-2026-111497.json